Report on Programming Task 1

Selma Kiefersauer

January 29, 2026

I decided to show just the results, plots and important parts of my python code in the report. The full python script can be seen on my GitHub page if in doubt. I hope this makes this easier to read and follow.

1 Scale heights

1.1 Scale-height at ground-level and 100km altitude

Task: calculate scale height at height = 0km and =100km for all species added together, calculated from the file MSIS.dat.

The equation for scaleheight:

$$H = \frac{k_{\mathrm{B}} T}{mg} \tag{1}$$

I assumed that a standard for gravitational acceleration at earth is g0 = 9.82 $\frac{m}{s^2}$. A formula for the variation of g is

$$g(z) = g(0) \frac{R_{\mathrm{E}}^2}{(R_{\mathrm{E}} + z)^2} \tag{2}$$

First I imported necessary libraries and defined constants.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

data = pd.read_csv("progrTasks/1atmosphere/MSIS.dat",sep=r"\s+",
                                skiprows=16)

##constants
k_B = 1.38 * 10e-24      #[m^2*kg/(K*s^2)] boltzmanns constant
g0 = 9.82 #[m/s] gravitational acceleration
R_E = 6378 *10e2 #[m] earth radius
p0 = 101325 #[Pa] std pressure at sealevel
#O weighs 16amu or g/mol
#N2 weighs 28amu or g/mol
#O2 weighs 32amu or g/mol
```

Then I defined a function to calculate the scale height from the ground up to 600km, and using the average mass of the particles at each height (step 1km). (The function can be used for constant g or g varying with height, for use in a later question).

```
#calculate scaleheight --> H= k_B*T/(m*g)
#scale height for all species
def calc_Hall(data_file, height, g):
     #nr density all molecules
    n = data_file.iloc[height,1]+ data_file.iloc[height,2]+
                                    data_file.iloc[height,3]
    #average molecule weight
    m = (data_file.iloc[height,1]*16 + data_file.iloc[height,2]*28
                                     + data_file.iloc[height,3]*
                                    32)/n *1.66054 * 10e-28 #[kg]
    #constant g or z-dependent g
    if g =="g_z":
        g = g0 * R_E**2 /(R_E + height*10e2)**2
    else:
        g = g0
    #scale height
    H = (k_B * data_file.iloc[height,5]) / (m * g)
    return H
```

This gives for z = 0km: 7.689km and for z=10km: 6.287km

1.2 Scaleheight for different species

I defined a new function that inputs the different species and also if one wants to use constant gravitational acceleration or g varying with height.

```
#scale height for single species, cst g or varying g
def calc_Hsingle(data_file, height, g, species):
    if species == "O":
        m = 16 * 1.66054 * 10e-28
    elif species == "N2":
        m = 28 * 1.66054 * 10e-28
    elif species == "O2":
        m = 32 * 1.66054 * 10e-28

    if g =="g_z":
        g = g0 * R_E**2 /(R_E + height*10e2)**2
    else:
        g = g0
    H = (k_B * data_file.iloc[height, 5]) / (m * g)
    return H
```

The results are presented in table 1 (also including the results from 1.1).

Table 1: scaleheight in km at different heights and for different species, using constant g ("g0") or varying g ("g(z)" in the calculations).

height (km)	all species	O	N2	O2
0 (g0)	7.689	-	-	-
10 (g0)	6.287	-	-	-
120 (g0)	-	21.147	12.084	10.573
120 (g(z))	-	21.950	12.543	10.975
600 (g0)	-	55.993	31.996	27.996
600 (g(z))	-	67.023	38.299	33.511

1.3 experimental scale height

I tried to implement the following equation for experimental scaleheight:

$$H_X = -\frac{n_X}{\partial n_X/\partial z} \tag{3}$$

I implemented the equation in the following way in python:

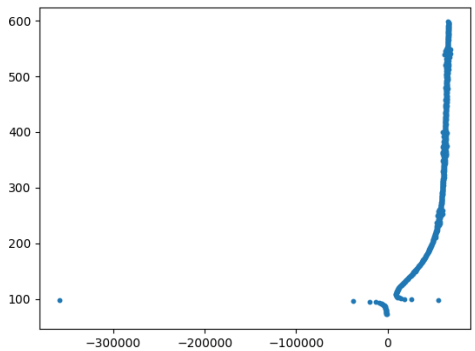
```
#experimental scale height H_x = -n_x/(dn_x/dz)
def exp_H(data_file, species):
    if species == "O":
        column = 1
    elif species == "N2":
        column = 2
    elif species == "O2":
        column = 3
    H = []
    for i in range(1,600):
        Hz = (-data_file.iloc[i,column])/((data_file.iloc[i,
                                          column]-data_file.iloc[i-
                                          1, column])/10e2)
        H.append(float(Hz))
    return H
```

This results are shown in table 2 and in figure 1.

Table 2: scaleheight in km at 120km and 600km and for different species.

height (km)	O	N2	O2
120	11.465	7.289	6.271
600	65.797	37.710	32.952

Figure 1: Experimental scaleheight.



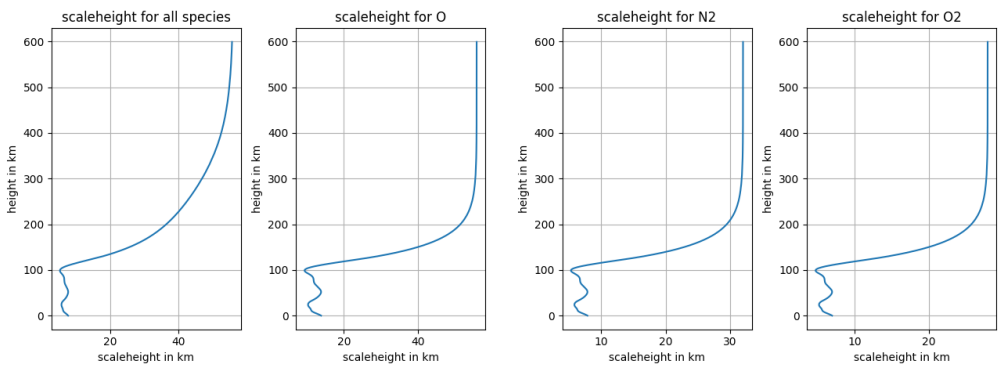
The scaleheight here is lower than the one calculated before, especially at 120km. At 600km the difference is less large. The plot shows that the calculation is not really stable, some values are way negative or in a weird range. It might be that I implemented the equation wrong or that I could be using a different way of calculating the discrete derivative that is more stable or accurate. This is something that I should probably still find out.

2 Atmospheric pressure variation

2.1 altitude variations of atmospheric scale-height

I used the equations above and the functions defined in 1.1. and 1.2. to plot the following altitude profiles (g = 9.82 m/s^2):

Figure 2: Altitude variations of scale height for different species.



2.2 pressure variations

The task is to calculate atmospheric pressure variation from the following equation:

$$\frac{\partial p}{\partial z} = -\frac{m(z)g(z)}{k_{\mathrm{B}}T(z)}p(z) \tag{4}$$

First by numerical integration and then by solving it (partly) analytically.

Numerical integration: the following function outputs an array with the pressure for all heights from 0 to 600km. (here dz = 1 because I already converted $H_{all}g_z$ into km when I stored the values for the scaleheight in the list!)

```
p_num = [0 for _ in range(len(H_all))]
p_num[0] = p0
dz = 1
for i in range(1,len(H_all)):
    p_num[i] = p_num[i-1] - (p_num[i-1]/H_all_gz[i-1])*(dz)
p_num = np.array(p_num)
```

Analytical solution: We need to integrate

$$\int_{p_0}^{p} \frac{1}{p} dp = -\int_{z_0}^{z} \frac{k_\mathrm{B} T(z)}{m(z)g(z)} dz \tag{5}$$

This gives the equation

$$p = p_0 exp(-\int_{z_0}^{z} \frac{k_\mathrm{B} T(z)}{m(z)g(z)} dz) \tag{6}$$

or as I already calculated the scaleheight earlier and want to reuse the results, in terms of scale height:

$$p = p_0 exp(-\int_{z_0}^{z} \frac{1}{H(z)} dz) \tag{7}$$

(also here dz = 1 for the same reason as above)

```
H_num = [0 for _ in range(len(H_all))]
H_num[0] = 1/H_all_gz[0]
for i in range(1, len(H_all)):
    H_num[i] = H_num[i-1] + 1/(H_all_gz[i-1])*dz
p_analytic = p0 * np.exp(-np.array(H_num))
```

The results are shown in figure 3, together with the plot from the ideal gas law, that is described in the next section. I also plotted the differences between the pressure profiles in a different plots, as they seem very similar and look like one and the same plot.

2.3 pressure profile from ideal gas law

If n is the number of particles and N is the number density we get for the ideal gas law that

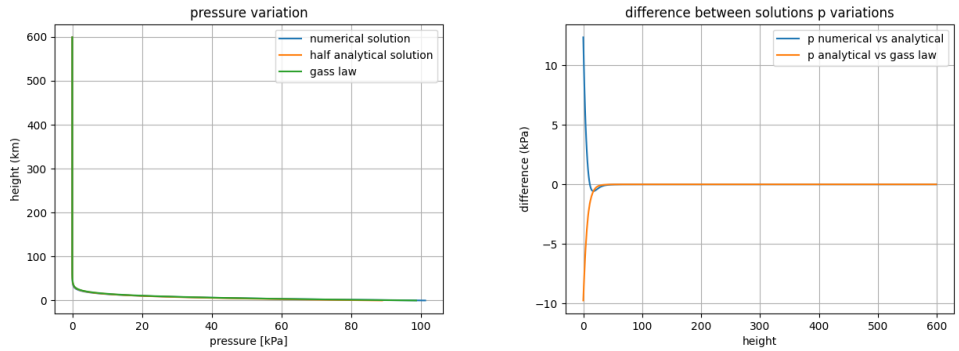
$$p = N k_\mathrm{B} T \tag{8}$$

Here we can use the number density from the datafile for N (after converting it to particles per cubic meter of course).

```
p_gasslaw = []
for i in range(600):
    n = (data.iloc[i,1]+data.iloc[i,2]+data.iloc[i,3])*10e5
    p_gasslaw.append(n*k_B*data.iloc[i,5])
p_gasslaw = np.array(p_gasslaw)
```

The pressure profiles and the difference to the other profiles is shown in the following plot:

Figure 3: Pressure variations calculated numerically, analytically and with the gas law and the difference between them.



They all give basically the same results,except for close to z=0, for the first iterations of the calculation.

2.4 Instabilities to convective turbulence

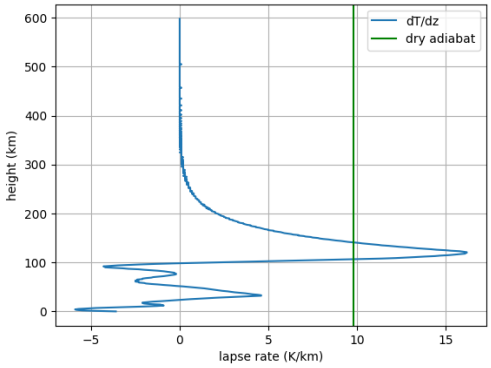
The instabilities might occur in areas where the change of temperature is larger than the lapse rate.

```
grad_T = []
for i in range(1,600):
    grad_T.append(data.iloc[i,5] - data.iloc[i-1, 5])
grad_T = np.array(grad_T)

plt.plot(grad_T, z[:-1], label="dT/dz")
plt.axvline(x=9.8, color = "green", label="dry adiabat")
plt.ylabel("height (km)")
plt.xlabel("lapse rate (K/km)")
plt.grid()
plt.legend()
plt.show()
```

The plot shows the change in temperature throughout the atmosphere and the dry adiabatic lapse rate at earth(9.8K/km).

Figure 4: Change in temperature and adiabatic lapse rate.



There seems to be an area of potential turbulence around 100-150km. This would be above the mesosphere, lower thermosphere. That‘s not what I would have expected, I thought there are more areas of instability closer to the ground.